

RAG Stack, Vector Databases & Retrieval

Embeddings • Vector Search • Chunking & Reranking



EMBED



STORE



RETRIEVE

Embeddings & Vector Search

WHAT'S AHEAD

Embeddings: generating vector representations of text

Vector databases and similarity search

Configuring a collection for cosine similarity

Generating, loading, and querying real reference documents

An Exact Key Doesn't Always Exist

PROBLEM

- Yesterday's tool call worked because a reservation ID is an exact key
- A guest asking “Is the restaurant pet-friendly?” has no ID to look up
- A SQL LIKE '%pet%' search fails if the policy says “animals welcome”
- Exact- and pattern-matching have no concept of meaning, only characters

“Is the restaurant pet-friendly?”

*"animals are welcome"
(no word 'pet' anywhere)*

?

WHERE id = ? / LIKE '%pet%' – both fail

No shared exact wording between the question and the policy text

Embeddings: Vector Representations of Meaning

SOLUTION

- An embedding: a fixed-length numeric vector generated from text
- Similar meaning → vectors close together, regardless of shared wording
- text-embedding-3-small (1536 dims) / -large (3072 dims), 8192-token limit
- Cosine similarity recommended — OpenAI vectors are normalized to length 1

HANDS-ON Generate embeddings for a set of capstone source documents.

"bring my dog..."



pet-related



checkout-related

Two unrelated topics stay apart; paraphrased meaning lands close together

A Python List Doesn't Scale

PROBLEM

- Comparing one query vector against every stored vector, by hand, in a loop
- Works for five documents; breaks down for hundreds, with no persistence
- No way to add, remove, or filter documents without recomputing everything
- The same reason a real application uses a database instead of a list

vector[0]

vector[1]

vector[2]

vector[3]

vector[4]

vector[5]

compared one at a time, every single query

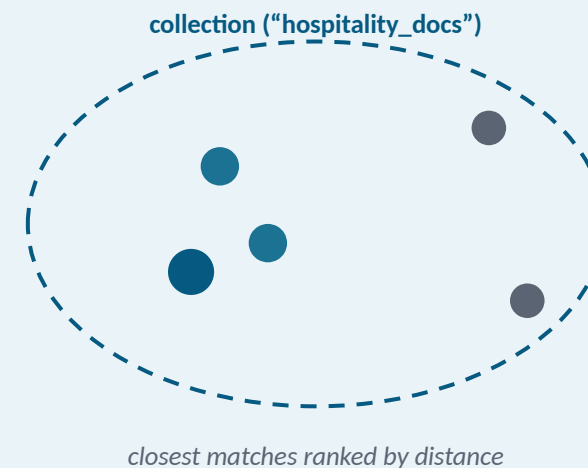
A linear scan through every stored vector, one at a time

Vector Databases & Similarity Search

SOLUTION

- A vector database stores embeddings + source text + metadata, indexed
- Chroma, run embedded (in-process, no server) — no vector-DB SaaS on the whitelist
- A collection's space config: l2 (Chroma's default), cosine, or ip
- `collection.query(query_texts=[...], n_results=k)` returns ranked matches

HANDS-ON Load the embeddings into a vector database; run a similarity search and inspect the results.



An indexed search returns only the nearest neighbors, ranked

Chunking, Retrieval & Reranking

WHAT'S AHEAD

Chunking: splitting documents to preserve retrieval quality

Retrieval pipelines: combining a retriever with a generator

Hybrid search and reranking, beyond similarity search alone

Assembling and evaluating the capstone's own RAG feature

One Vector Can't Represent a Whole Document

PROBLEM

- A real policy document covers many topics in one file, not one sentence
- Embedding it whole blurs every topic into one averaged vector
- The same “one prompt can't do a two-part job” problem, now at document scale
- A hard ceiling too: embedding models cap input at 8192 tokens per call

one long, multi-topic document



Four topics compressed into one blurred vector

Chunking Strategies with RecursiveCharacterTextSplitter

SOLUTION

- Chunking: split a document into focused pieces before embedding each one
- Fixed-size cuts can slice a sentence in half at an arbitrary boundary
- RecursiveCharacterTextSplitter: tries separators, paragraph → sentence → word
- chunk_size (max length) and chunk_overlap (repeated boundary text) are trade-offs

HANDS-ON Apply a chunking strategy to a long document and compare retrieval quality before and after.



five focused chunks → five focused vectors

Each chunk now embeds as its own focused, single-topic vector

A Similarity Search Returns Text, Not an Answer

- Even a well-chunked similarity search only returns ranked raw passages
- A guest doesn't want three policy excerpts, they want an actual reply
- Handing back raw retrieved text defeats the point of an assistant
- Risk: a model answering with no grounding can quietly guess, not know

PROBLEM

chunk: pets

chunk: checkout

chunk: cancel

?

no reply written for the guest

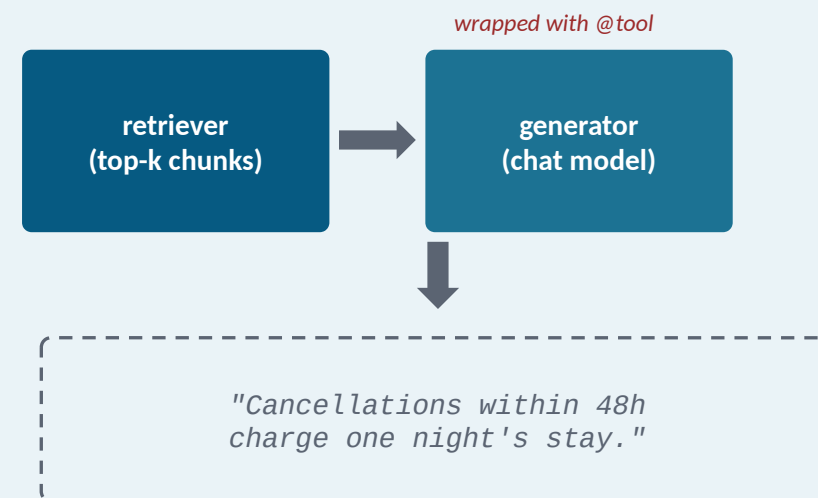
Three ranked chunks, and nowhere for them to go

Retrieval Pipelines: Retriever + Generator

SOLUTION

- RAG: a retriever (Module 16A's work) plus a generator (the chat model)
- `vectorstore.as_retriever(search_kwargs={"k": 3})` — a standard, callable retriever
- Retrieved chunks feed a prompt: “answer using ONLY the context below”
- Wrapped with `@tool`, exactly like yesterday's `get_reservation`

HANDS-ON Assemble a full RAG pipeline, retriever plus generator, as the capstone's RAG feature.



Retrieved chunks in, one grounded reply out

A Cheap Ranking Can Get the Top Results Wrong

PROBLEM

- Similarity search compares vectors independently, fast but shallow
- An exact term (a rate-plan code, a proper noun) can lose to a vaguer match
- The ranking never actually looks at query and candidate together
- A single dense signal can't catch a keyword match an embedding blurs past

spa cancellation

general cancellation



Two candidates land almost equally close — which one is actually right?

Hybrid Search and Reranking

SOLUTION

- Hybrid search: dense (meaning) + sparse/keyword (BM25), fused with RRF
- Chroma Cloud has this built in — self-hosted Chroma doesn't; hand-assemble it
- Reranking: a second, careful pass over an already-narrowed top-k
- Today's reranker: the chat model, scoring 0–10 with Module 12B's pattern

HANDS-ON Add a reranking step and compare its top results against the plain similarity search, using a small query set.



The reranker separates two near-identical candidates correctly

Evaluating the Difference: A Small Query Set

- Run the same candidates through plain similarity search and LLM reranking
- Compare each query's top-3 lists side by side, not just one test case
- Look for the query where the two orderings actually disagree
- That disagreement is the concrete, explainable proof reranking did something

HANDS-ON Add a reranking step and compare its top results against the plain similarity search, using a small query set.

Plain search	LLM reranked
1. checkout	1. checkout
2. cancellation	2. spa hours
3. spa hours	3. cancellation

One reordered row is the evidence reranking changed the outcome